Telco Customer Churn Dataset

1. Import required libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

2. Load the dataset

```
print("Upload WA_Fn-UseC_-Telco-Customer-Churn.csv")
uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(filename)

print("\nFIRST 10 RECORDS")
display(df.head(10))

print("\nLAST 10 RECORDS")
display(df.tail(10))

print("\nSHAPE OF DATASET:", df.shape)

print("\nCOLUMN NAMES:")
print(df.columns.tolist())
```

```
Upload WA_Fn-UseC_-Telco-Customer-Churn.csv
Choose Files archive.zip
archive.zip(application/x-zip-compressed) - 175758 bytes, last modified: 10/9/2026 - 100% done
Saving archive.zip to archive.zip

FIRST 10 RECORDS
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurit
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	N
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Ye
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Ye
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Ye
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	N
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	N
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	N
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	Ye
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	N
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	Ye

10 rows × 21 columns

```
LAST 10 RECORDS
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecu
7033	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic	
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic	
7035	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic	
7036	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL	
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No	No in se
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	

10 rows × 21 columns

```
SHAPE OF DATASET: (7043, 21)

COLUMN NAMES:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetServic
```

3. Explore the dataset

```
print("\nDATASET INFORMATION:")
df.info()

print("\nSUMMARY STATISTICS:")
display(df.describe(include="all"))
```

```
print("\nMISSING VALUES:")
print(df.isnull().sum())

print("\nNUMBER OF DUPLICATE RECORDS:")
print(df.duplicated().sum())
""
```

```
DATASET INFORMATION:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

SUMMARY STATISTICS:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	Onl
count	7043	7043	7043.000000	7043	7043	7043.000000	7043	7043	7043	
unique	7043	2	NaN	2	2	NaN	2	3	3	
top	3186-AJIEK	Male	NaN	No	No	NaN	Yes	No	Fiber optic	
freq	1	3555	NaN	3641	4933	NaN	6361	3390	3096	
mean	NaN	NaN	0.162147	NaN	NaN	32.371149	NaN	NaN	NaN	
std	NaN	NaN	0.368612	NaN	NaN	24.559481	NaN	NaN	NaN	
min	NaN	NaN	0.000000	NaN	NaN	0.000000	NaN	NaN	NaN	
25%	NaN	NaN	0.000000	NaN	NaN	9.000000	NaN	NaN	NaN	
50%	NaN	NaN	0.000000	NaN	NaN	29.000000	NaN	NaN	NaN	
75%	NaN	NaN	0.000000	NaN	NaN	55.000000	NaN	NaN	NaN	
max	NaN	NaN	1.000000	NaN	NaN	72.000000	NaN	NaN	NaN	

11 rows × 21 columns

```
MISSING VALUES:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

NUMBER OF DUPLICATE RECORDS:
0
''
```

4. Clean the data

```
ddf = df.drop_duplicates().copy()

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)


numerical_cols = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_cols = df.select_dtypes(
    include="object"
).columns.tolist()

print("\nMISSING VALUES BEFORE CLEANING:")
print(df.isnull().sum())

print("\nDUPLICATES BEFORE CLEANING:")
print(df.duplicated().sum())

for col in numerical_cols:
    df[col] = df[col].fillna(df[col].median())


for col in categorical_cols:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nMISSING VALUES AFTER CLEANING:")
print(df.isnull().sum())

print("\nDUPLICATES AFTER CLEANING:")
print(df.duplicated().sum())
```

```
MISSING VALUES BEFORE CLEANING:
customerID           0
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges        11
Churn                0
dtype: int64

DUPLICATES BEFORE CLEANING:
0

MISSING VALUES AFTER CLEANING:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
DUPLICATES AFTER CLEANING:
0
```

5. Encode categorical columns

```
binary_cols = [
    "gender", "Partner", "Dependents", "Churn"
]

for col in binary_cols:
    if col in df.columns:
        encoder = LabelEncoder()
        df[col] = encoder.fit_transform(
            df[col].astype(str)
        )

multi_cols = [
    "Contract", "PaymentMethod", "InternetService"
]

df = pd.get_dummies(
    df,
    columns=[col for col in multi_cols if col in df.columns],
    dtype=int
)

print("\nDATA AFTER ENCODING:")
display(df.head(10))
```

```
DATA AFTER ENCODING:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup
0	7590-VHVEG	0	0	1	0	1	No	No phone service	No	Yes
1	5575-GNVDE	1	0	0	0	34	Yes	No	Yes	No
2	3668-QPYBK	1	0	0	0	2	Yes	No	Yes	Yes
3	7795-CFOCW	1	0	0	0	45	No	No phone service	Yes	No
4	9237-HQITU	0	0	0	0	2	Yes	No	No	No
5	9305-CDSKC	0	0	0	0	8	Yes	Yes	No	No
6	1452-KIOVK	1	0	0	1	22	Yes	Yes	No	Yes
7	6713-OKOMC	0	0	0	0	10	No	No phone service	Yes	No
8	7892-POOKP	0	0	1	0	28	Yes	Yes	No	No
9	6388-TABGU	1	0	0	1	62	Yes	No	Yes	Yes

10 rows × 28 columns

6. Standardise numerical features

```
scale_cols = [
    "tenure", "MonthlyCharges", "TotalCharges"
]

scaler = StandardScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])

print("\nDATA AFTER STANDARDISATION:")
display(df[scale_cols].head(10))
```

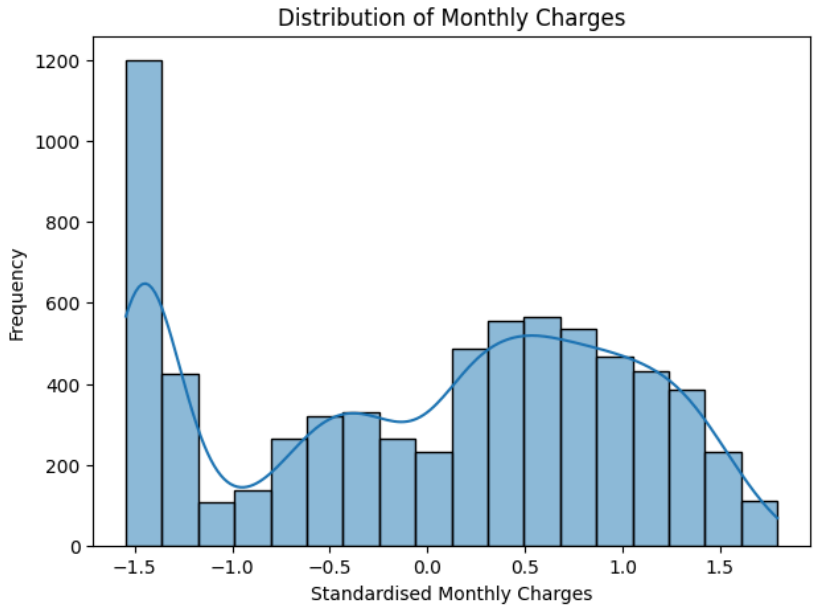
```
DATA AFTER STANDARDISATION:
```

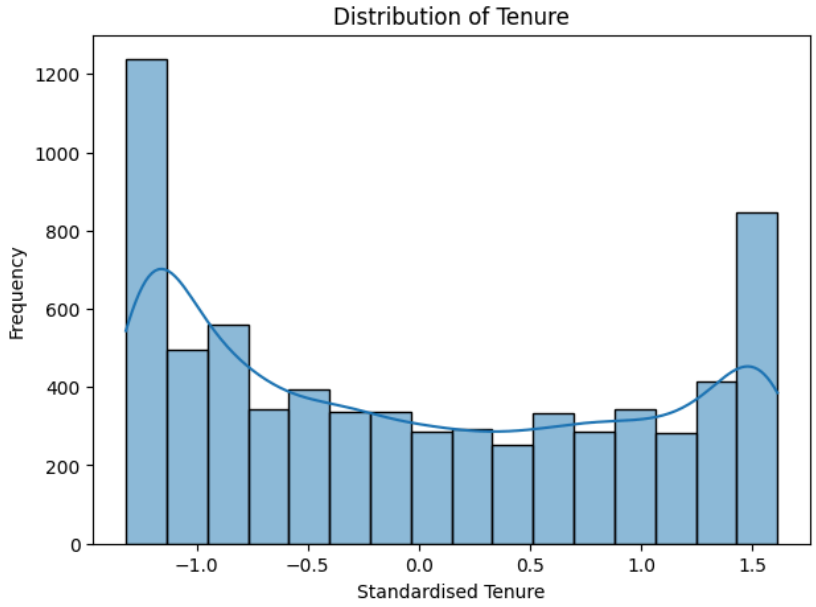
	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.194766
4	-1.236724	0.197365	-0.940470
5	-0.992402	1.159546	-0.645186
6	-0.422317	0.808907	-0.146799
7	-0.910961	-1.163647	-0.874137
8	-0.177995	1.330711	0.337349
9	1.206498	-0.286218	0.532439

7. Distribution of MonthlyCharges

```
plt.figure(figsize=(7, 5))
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of Monthly Charges")
plt.xlabel("Standardised Monthly Charges")
plt.ylabel("Frequency")
plt.show()

# Distribution of Tenure
plt.figure(figsize=(7, 5))
sns.histplot(df["tenure"], kde=True)
plt.title("Distribution of Tenure")
plt.xlabel("Standardised Tenure")
plt.ylabel("Frequency")
plt.show()
```





8. Count plot: Contract vs Churn

```
plot_df = pd.read_csv(filename)
plot_df["TotalCharges"] = pd.to_numeric(
    plot_df["TotalCharges"], errors="coerce"
)
plot_df = plot_df.drop_duplicates()

plt.figure(figsize=(8, 5))
sns.countplot(
    data=plot_df,
    x="Contract",
    hue="Churn"
)
plt.title("Contract Type vs Customer Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.show()
```